Technical report: Booklet NLI, claim checking over Swiss voting booklets with Apertus

- **Track:** Track 2A, OST challenge: *Multilingual Natural Language Inference over Swiss Official Voting Booklets*
- **Event:** Online
- **Team:** kaiser-data (Martin Kaiser)
- **Code:** https://github.com/kaiser-data/hack-apertus-booklet-nli (`track_2a/`)
- **Visual summary:** `track_2a/docs/results.html`

1. Summary

Given an official Swiss federal voting booklet (German, French or Italian PDF) and a claim in any of the three languages, the system decides **entailment, neutral or contradiction** and cites the booklet passages, with page numbers, that the decision rests on. A small multilingual embedder picks the 16 passages closest to the claim; **Apertus-v1.5-8B** reads them and returns its label, cited passages and a short reason as structured JSON. Apertus is the only model that makes the decision.

On the held-out test split (976 claims, 14 vote dates never used during development), the system reaches **macro-F1 0.891** (95% bootstrap interval 0.871–0.909) with about **3,400 prompt tokens** and a median **4.5 s** Apertus response per claim (PDF parsing and retrieval excluded) on one L40S GPU. On a 60-claim test subsample, giving Apertus the whole booklet instead of retrieved passages scores **macro-F1 0.735 against 0.861** for retrieval on the same claims, while using **9× the prompt tokens** (31,400 vs 3,450). Six of the whole-booklet answers never finish: the model starts listing passage numbers in sequence and never reaches the label. Doubling the output budget does not fix this, and a second whole-booklet run scored 0.771. The honest summary: retrieval is at least as accurate as reading the whole booklet, more reliable, at a ninth of the tokens and about a third of the time.

2. Architecture

```
booklet PDF ─► passages of ~900 chars, with page + outline path    (CPU, cached)
claim ──────► top-16 passages by cross-lingual embedding           (CPU, cached per booklet)
           ─► Apertus-v1.5-8B: {reason, evidence ids, label}       (GPU, vLLM, JSON schema)
```

1. **Passages** (`booklet.py`). PyMuPDF extracts text page by page. Soft hyphens and words broken across lines are repaired. PDF text blocks are merged into passages of about 900 characters that never cross a page (a single long block is kept whole, so 195 of 7,610 dev+test passages are longer; the longest has 1,623 characters). Each passage keeps its page number and the PDF outline path of that page (vote › part › subsection), so the model and the user can see where a passage comes from. Parsed passages are cached per booklet.
2. **Retrieval** (`retrieve.py`). Claim and passages are embedded with `paraphrase-multilingual-MiniLM-L12-v2` (ONNX via fastembed, CPU). Because the embedding space is shared across languages, a French claim retrieves German passages directly, with no translation step. The top k passages are passed in document order.
3. **Verdict** (`nli.py`, `llm.py`). Apertus gets a system prompt with the three label definitions (including “a statement attributed to the wrong actor is a contradiction”), the numbered passages and the claim. Output is constrained by a JSON schema (`reason`, `evidence`, `label`); if a server does not support structured output, the client falls back to plain JSON parsing. Output is capped at 300 tokens; an answer cut off by the cap cannot be parsed. Unparseable answers count as neutral and are reported as parse failures.
4. **Interfaces** (`cli.py`). `predict` (one claim), `batch` (JSONL), `eval` (resumable; writes predictions and metrics), `retrieval` (recall@k without an LLM). `make run` runs `eval` in Docker.

Everything that touches the GPU sits behind an OpenAI-compatible endpoint, configured by `LLM_NAME`, `LLM_BASE_URL` and `LLM_API_KEY`.

3. Use of Apertus

- **Model:** `swiss-ai/Apertus-v1.5-8B` (official weights, bf16, no fine-tuning, no quantisation).
- **How it is used:** inference: zero-shot NLI with evidence selection.
- **Where it runs:** Modal, one NVIDIA L40S, swiss-ai’s own vLLM build for Apertus 1.5 (`ghcr.io/swiss-ai/vllm_apertus_1.5_release`, pinned by digest), 128k context, prefix caching, scale-to-zero after 5 idle

minutes (`modal/serve_apertus.py`). Temperature 0.
- **Evaluation runner:** `modal/run_eval.py` runs `booklet-nli eval` in a CPU container built from the project's Dockerfile, next to the GPU server, so no evaluation load runs on a laptop.

The prompt is in `src/booklet_nli/nli.py` (`SYSTEM`). It is written in English; claims and passages stay in their original languages, and the model answers its `reason` field in whichever language it chooses.

4. Data

- **Dataset:** `OSTswiss/MNLIoverSwissVotingBooklets` v1.1 (MIT licence), 1,488 claim/booklet pairs, balanced over the three labels and over all nine claim × booklet language pairs.
- **Booklets:** official explanations of the Federal Council, downloaded from bk.admin.ch on first use and cached locally; not redistributed in the repository.
- **Split.** The dataset ships without a split. We split **by vote date** (hash of the booklet publication date), so the German, French and Italian booklets of one vote are always on the same side and no booklet is seen during development and then tested on: dev = 512 pairs / 6 vote dates (all design decisions), test = 976 pairs / 14 vote dates (run once).
- No personal data is involved: claims and booklets are public political texts.
- **Sources and size we chose not to add.** We did not fine-tune (a LoRA on 512 dev rows from six vote dates would mostly learn those dates). We did not add XNLI or ANLI: both are CC BY-NC 4.0, XNLI has no Italian, and both are sentence-pair NLI rather than document-grounded. We did not add legal texts (Fedlex) or parliamentary debates as evidence: the task asks whether the *booklet* settles a claim, and a statute that entails the claim would turn a correct "neutral" into a wrong "entailment". The dataset's `vote` column and `baseline_score` are never used as model input.

5. Evaluation

Metrics. Macro-F1 over the three labels (primary), accuracy, per-language F1. **Evidence hit**: share of non-neutral claims where at least one cited passage overlaps the annotated reference snippet (≥ 50 % of the passage's word 5-grams occur in the reference). **Context recall**: share of non-neutral claims where the reference passage was among those given to the model. Prompt and completion tokens and wall-clock seconds per claim (client side, including network). **Uncertainty**: 95% percentile intervals from 2,000 bootstrap resamples of claims (seed 0); comparisons between two runs use a paired bootstrap on the claims both runs share (`booklet-nli compare`).

5.1 Main result (test split, held out, run once)

Setup	n	Macro-F1 (95% CI)	Acc	Evidence hit	Prompt tokens (mean)	s/claim (median)	Parse fail
Full booklet (baseline)	60	0.735 (0.61–0.84)	0.733	0.548	31,402	52.4*	6
Full booklet, rerun with a 600-token cap	60	0.771 (0.65–0.87)	–	–	31,402	61.9*	6
RAG, k=16, same 60 claims	60	0.861	0.867	0.690	3,446	4.4	1
RAG, k=16 (ours)	976	**0.891** (0.871–0.909)	0.891	0.716	3,424	4.5	2

On the 60 shared claims, retrieval gets 11 claims right that the full booklet gets wrong, and the reverse happens 3 times. The paired difference is +0.127 (95% CI +0.012 to +0.252), but 5 of those 11 are full-booklet parse failures, which count as neutral.

Why whole-booklet answers fail. Every parse failure in both runs stopped at exactly the output cap. Inspecting one raw answer shows why: after a sensible `reason`, the `evidence` list runs 150, 151, 152, … through consecutive passage numbers, so the `label` field, last in our schema, is never written. Raising the cap from 300 to 600 tokens and rerunning the 60 claims gave 6 failures again (5 of them the same claims, now stopping at 600). This is a failure mode

of long input, not of the budget. It can be scored two ways: counting failures as neutral, as the system does, retrieval leads by +0.127 (first run) or +0.091 (95% CI −0.004 to +0.197, rerun); excluding the 6 failed claims of the first run, by +0.053 (95% CI −0.056 to +0.168). Retrieval therefore scores higher on this sample, though the second interval includes zero, answers all of those claims (its one failure in the 60 is a different claim; 2 in 976 overall), and is clearly cheaper and faster. The 60 claims were drawn with a fixed seed and are not exactly balanced (24 entailment, 18 neutral, 18 contradiction).

* Full-booklet latency is inflated by load: 12 concurrent requests of ~31k tokens share one GPU (the rerun also shared it with a dev run). On dev with the same setting it was 15.5 s median, still about 3× slower than retrieval.

Confusion matrix, test, RAG k=16 (rows gold, columns predicted):

	entailment	**neutral**	**contradiction**
entailment	292	21	22
neutral	1	305	18
contradiction	13	31	273

By language (macro-F1): booklet DE 0.866, FR 0.898, IT 0.908; claim DE 0.883, FR 0.899, IT 0.890; cross-lingual pairs 0.899 (n = 641), same-language pairs 0.875 (n = 335). Cross-lingual pairs score no worse than same-language ones (difference +0.024, 95% CI −0.017 to +0.071), so we see no evidence that a translation step is needed.

Mean completion length is 86 tokens. The 90th-percentile latency is 7.1 s over all 976 claims (6.7 s over the 940 claims run on Modal, see §6).

5.2 Design decisions (dev split)

All choices were made on dev, on the same 150-claim stratified sample. **Order of work:** k and the prompt were fixed on dev (this section); the test split was then run once (§5.1); the data experiments in §5.3 came afterwards and used dev only.

Variant	**Macro-F1 (95% CI)**	**Context recall**	**Prompt tokens**	**s/claim (median)**
RAG, k=8	0.850 (0.79–0.90)	0.82	1,818	3.0
RAG, k=16 (chosen)	**0.890** (0.84–0.94)	0.92	3,354	5.0
RAG, k=16 + extra “contradiction, not neutral” rule in the prompt	0.864 (0.80–0.92)	–	3,404	4.7
Full booklet (n = 30)	0.582 (0.39–0.74)	1.00	31,785	15.5

- **Retrieval depth.** Retrieval recall@k of the reference passage on dev (all 338 non-neutral claims): @1 0.31, @3 0.56, @5 0.68, @8 0.77, @12 0.85, @16 0.91, @20 0.93. (The context recall column above is the same quantity measured on the 150-claim sample inside each run.) Going from k=8 to k=16 fixed 9 claims and broke 3, at about 1.8× the tokens. The paired difference is +0.041 (95% CI −0.005 to +0.091), so the F1 difference alone does not prove k=16 better; the choice rests on it together with the recall gain from k=8 to k=16 (0.77 → 0.91). The later test score (0.891) is consistent with dev (0.890) but played no part in the choice.
- **A rejected prompt change.** The most frequent dev error was contradiction predicted as neutral. One added sentence telling the model to prefer contradiction when the passages cover the point made things worse (fixed 2, broke 6; contradiction → neutral went from 7 to 9; paired difference −0.027, 95% CI −0.066 to +0.011), so it was reverted. The same change flipped 11 of 150 predictions at temperature 0, which also shows how large the noise is at n = 150: differences of a few claims on dev are not meaningful.
- **Rerun noise.** Rerunning the chosen setting with only the output cap raised to 600 tokens (no answer reached it; the longest was 225 tokens) changed 5 of 150 predictions and moved dev macro-F1 from 0.890 to 0.864. Four of the five had prompts of identical length, so their variation comes from the serving stack (batched GPU inference is not bit-exact at temperature 0); the fifth had a different passage set (4,177 vs 3,744 prompt tokens). Across all 150 claims, 42 prompts differed in length between the two runs: near-tied passages were ranked differently (see §7), so part of the rerun gap is retrieval variation, not only GPU nondeterminism. The paired bootstrap for this pair is +0.027 (95% CI +0.005 to +0.056): it resamples claims but cannot see run-to-run

variation, so its intervals are too narrow for comparing two runs. The k=8 vs k=16 difference (+0.041) is only about 1.5 times this rerun gap, and the test score 0.891 is a single run; one rerun on dev differed by 0.027.

- **Embedder.** MiniLM was chosen because it was validated end to end and runs on CPU; `multilingual-e5-large` could not be benchmarked on the 8 GB development laptop.

5.3 Data-side experiments (dev, not adopted)

Two retrieval changes were tested on dev after the test run, both suggested by external review. Recall@k is over all 338 dev claims with a reference passage; the end-to-end row uses the same 150 claims as §5.2.

Variant	Recall@8	Recall@16	Recall@20	Dev macro-F1 (n = 150)
Parser v2 (used for all reported results)	0.769	0.911	0.929	0.890
v4: split oversized blocks at sentences, keep short blocks (rates, table cells), ≤150-char in-page overlap	**0.808**	**0.926**	**0.950**	0.878
v3 + route retrieval to the best-matching vote section (top-level outline entry)	0.760	0.823	0.837	–

- **Better passage cutting** finds the reference passage more often, but end-to-end macro-F1 did not improve (paired difference −0.013, 95% CI −0.072 to +0.050; 8 claims fixed, 10 broken), and evidence hit moved only from 0.707 to 0.727. Retrieval recall is not the binding constraint at k = 16. The code is kept on the branch `experiment/parser-v4`.
- **Vote routing** (score each vote section by its three best passages, fill most of the context from the winner, keep the four best global passages as a fallback) clearly hurt recall: the section choice was wrong too often, and global ranking already concentrates on the right vote. It was not run end to end, and its code was not kept.
- Neither change was run on test, so the test result in §5.1 remains a single, untouched run.

5.4 Error analysis (dev, k=16, 16 errors)

- 4 contradiction → neutral and 2 entailment → neutral with the right passage in context: the model reads “the passages do not mention X” where the passage states X or something incompatible with it.
- 5 retrieval misses. Three are the same claim in three languages about a transition period for farmers, where retrieval returned passages about the AHV pension reform, another vote in the same multi-vote booklet. Two concern a VAT rate change (7.7 → 8.1 %) whose passage was not retrieved.
- The remaining 5 errors: 3 contradiction → entailment and 2 neutral → contradiction, all with the reference passage in context.

6. Limitations

- **Retrieval misses.** 70 of 652 non-neutral test claims (10.7 %; 7.2 % of all claims) never see their annotated reference passage (context recall 0.89). This is not a hard ceiling: 44 of those 70 were still judged correctly, from other passages that carry the same information. Missed references still account for 26 of the 106 test errors.
- **Multi-vote booklets.** One booklet covers several votes; retrieval can return passages of the wrong vote. Routing by the outline’s top-level section made this worse (§5.3).
- **Contradiction vs neutral** remains the main confusion (31 test claims), and prompt wording did not fix it.
- **Evidence hit is a proxy.** It measures n-gram overlap with one annotated snippet. A correct but different passage counts as a miss, and a cited passage can overlap the snippet without being the one that decides the claim, so 0.72 can be off in either direction. It is not a manual audit of citations.
- **Latency** is measured client side over the internet against a serverless GPU, with 12 concurrent requests. A cold start (about 5 minutes) inflates the first requests of a run, which is why we report medians; the per-claim seconds in `predictions.jsonl` include it. The first 36 test claims were run from a heavily loaded laptop before the run moved to Modal; over the 940 claims run on Modal the median is 4.47 s and the mean 8.71 s.

- **Baselines.** The full-booklet comparison uses a 60-claim test subsample, not all 976 claims, to keep GPU cost low, and its size depends on how runaway answers are scored (§5.1); it is directional, not precise.
- **Single run, few dates.** Each configuration was run once at temperature 0, and an identical rerun on dev moved macro-F1 by 0.027 (§5.2). The bootstrap intervals resample claims, but not runs, and the test set comes from only 14 vote dates, so claims are not fully independent; dev decisions rest on 6 dates.

7. Reproducibility

- **Hardware:** Apertus on one NVIDIA L40S (Modal); retrieval and evaluation on a 4-vCPU Modal container; development on an 8 GB Apple M2.
- **Software:** Python 3.12, dependencies pinned in `uv.lock`; vLLM image pinned by digest.
- **Data:** dataset pinned to Hugging Face revision `fc2b276` and checked by SHA-256 on download (`data.py`).
- **Determinism:** temperature 0, sample seed 0, split by hash of vote date. Embeddings computed on different CPUs can differ in the last digits and change the order of near-tied passages (seen once in a 3-claim check: same label, 4,548 vs 4,340 prompt tokens).
- **Docker check:** `make run LIMIT=3` from the repository root, on a clean arm64 machine (Jetson Orin, Docker 29) against the Modal endpoint, built the image and reproduced the reported test predictions for those 3 claims exactly (same labels, same prompt token counts).
- **Commands:**

```
make run                                    # Docker: 60-claim test sample (seed 0)
make run LIMIT=976 RUN=runs/test-fresh      # full test split, fresh (without RUN it would resume
                                            # the committed run and only recompute its metrics)
modal deploy modal/serve_apertus.py         # serve Apertus
modal run modal/run_eval.py::main --split test --limit 976 --k 16 --name test-rag-k16-n976
```

The `config` block in `runs/dev-full-n30/metrics.json` names `multilingual-e5-large` as embedder; full-booklet mode does not embed, so that field is unused there.

All run directories (`predictions.jsonl` with label, cited pages, reason, tokens and seconds per claim; `metrics.json`) are in `track_2a/runs/`. A visual summary of the results is in `track_2a/docs/results.html`.

8. Next steps

- Disambiguate the vote with a learned signal instead of outline routing: e.g. let Apertus pick the vote from the booklet's table of contents in a cheap first call, then retrieve within it.
- A cross-encoder re-ranker over the top 30 passages, to raise recall at k ≤ 16 and cut tokens.
- Two-pass verification for claims predicted neutral: ask Apertus to look for an incompatible statement explicitly, only when a passage is close in embedding space.
- Stop runaway evidence lists: put `label` before `evidence` in the schema and cap the list (e.g. `maxItems: 5`), then measure on dev with repeated runs.
- Run a larger Apertus model on the same pipeline to see whether the contradiction/neutral confusion is a capacity issue.
- A small web front end: paste a claim, see the verdict with highlighted booklet pages.

License

Code: Apache-2.0. This report: Creative Commons Attribution 4.0 (CC-BY-4.0).

References

- Swiss AI Initiative. *Apertus-v1.5-8B.* https://huggingface.co/swiss-ai/Apertus-v1.5-8B
- OST. *MNLI over Swiss Voting Booklets*, v1.1. https://huggingface.co/datasets/OSTswiss/MNLIoverSwissVotingBooklets
- Reimers & Gurevych (2020). *Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation.* EMNLP. (`paraphrase-multilingual-MiniLM-L12-v2`)
- Kwon et al. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention.* SOSP. (vLLM)

- Swiss Federal Chancellery. *Erläuterungen des Bundesrates* / *Explications du Conseil fédéral* / *Spiegazioni del Consiglio federale.* https://www.bk.admin.ch